NAME: SIVARANJANII.M
ROLL No.:241901109

EXPERIMENT 7

PERFORM WEB APPLICATION RECONNAISSANCE USING VARIOUS TOOLS

AIM:

To gather information about the structure, technologies, and possible entry points of a target web application before testing it for vulnerabilities.

THEORY:

Web application reconnaissance is different from web *server* reconnaissance (Experiment 5/6) — here the focus shifts from the underlying server software to the **application itself**: its pages, input forms, cookies, technologies (frontend/backend frameworks), and how users interact with it.

Key things to look at:

- Application technology stack (PHP, ASP.NET, Node.js, React, etc.)
- Login forms, search boxes, and file upload forms (common attack surfaces)
- Cookies set by the application and their security flags (HttpOnly, Secure)
- Client-side JavaScript files (may contain API endpoints or hardcoded secrets)
- Error messages (verbose errors can reveal backend details)

TOOLS REQUIRED

- Burp Suite (Community Edition) — for intercepting and inspecting traffic
- WhatWeb / Wappalyzer browser extension
- Web browser Developer Tools
- Target: DVWA or OWASP Juice Shop

PROCEDURE:

Step 1: Identify the Technology Stack

Run WhatWeb against the application:

```
whatweb http://<target-ip>/dvwa
```

Note the CMS/framework, programming language, and any JavaScript libraries detected

(older library versions are worth noting for later vulnerability checks).

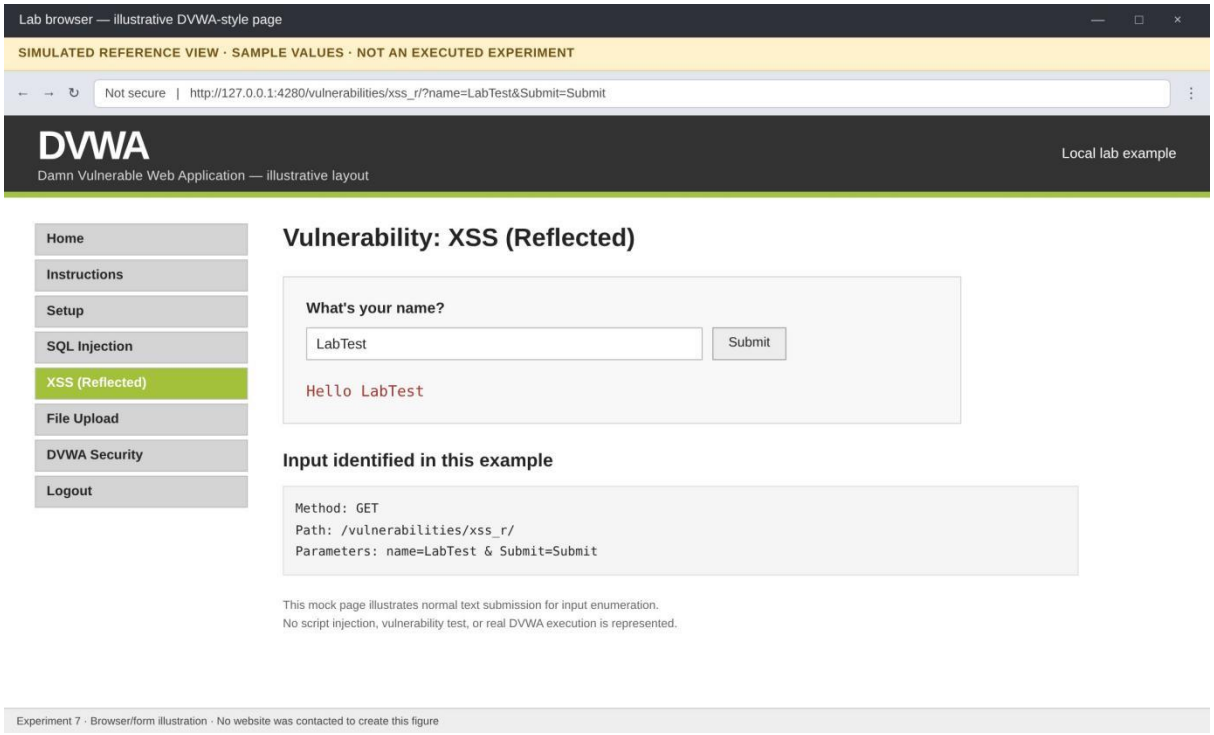


Fig. 1: Application form and input parameters (simulated reference).

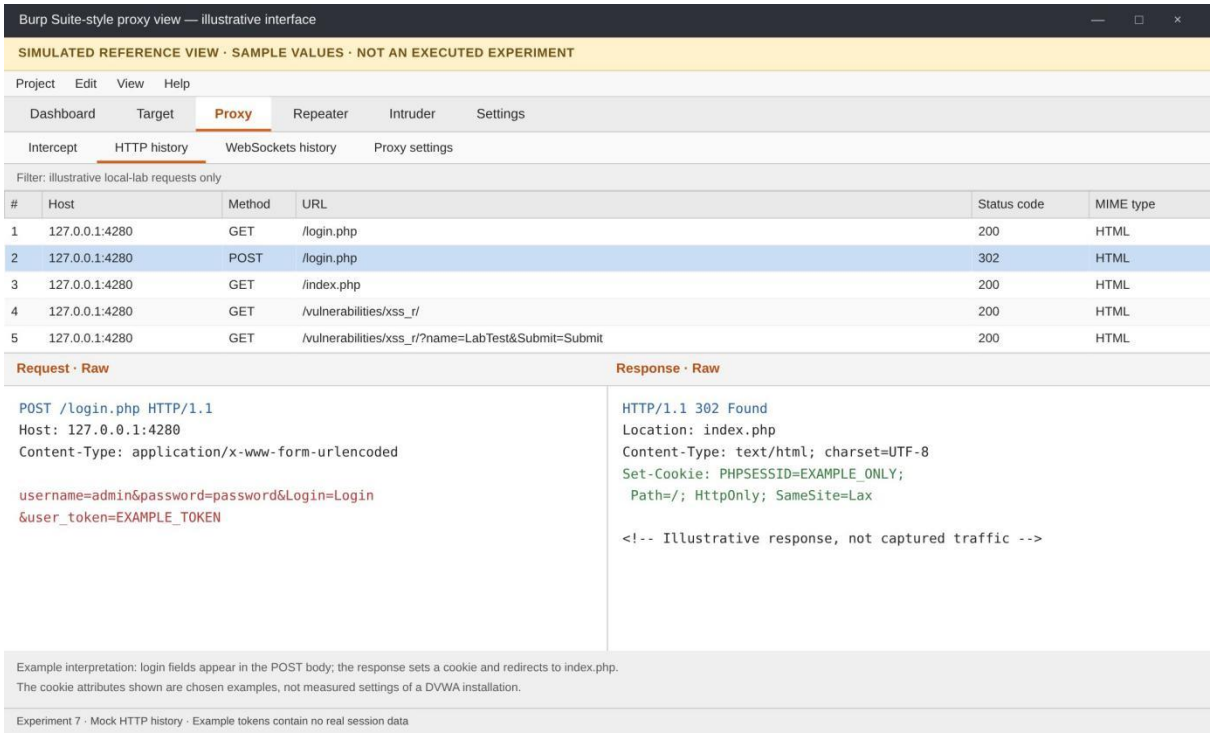


Fig. 2: HTTP request and cookie inspection (simulated reference).

Step 2: Set Up Burp Suite as a Proxy

1. Open Burp Suite → go to the **Proxy** tab → make sure "Intercept" is set up on 127.0.0.1:8080.
2. In Firefox, set the network proxy to 127.0.0.1, port 8080(Settings → Network Settings → Manual proxy).
3. Browse to the target application through Firefox; Burp will now capture every request/response.

Step 3: Explore the Application Manually While Capturing Traffic

Click through the target application (login page, search box, profile page) while Burp's **HTTP History** tab records every request. Look for:

- Hidden form fields (<input type="hidden">)
- Parameters passed in the URL (e.g., ?id=3)
- Any API calls made in the background (visible as JSON requests/responses)

Step 4: Inspect Cookies

In Burp's HTTP History, click on any request/response pair and check the Set-Cookie header. Note whether flags like HttpOnlyand Secureare present:

```
Set-Cookie: PHPSESSID=abc123; path=/
```

Missing HttpOnlymeans the cookie can be read by JavaScript (relevant for XSS-based session theft, covered in Experiment 12).

Step 5: Review Client-Side JavaScript Files

In the browser Developer Tools → **Sources/Debugger** tab, open each .jsfile loaded by the page and search for:

- Hardcoded API keys or URLs
- Comments left by developers
- Hidden/unused endpoints not linked anywhere on the visible site

Step 6: Trigger and Study Error Messages

Enter unexpected input into a form field (e.g., a single quote 'in a search box) and observe the error message returned. A verbose error like a full SQL error string can reveal the backend database type - useful information for later testing .

OUTPUT:

A structured recon report of the web application: technology stack used, all discovered input points (forms/parameters), cookie security flags, and any information leaked through JavaScript files or error message